โครงการอบรมโอลิมปิกวิชาการ ค่าย 2 สาขาคอมพิวเตอร์
วันที่ 27 มี.ค. – 10 เม.ย. 2569

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยเชียงใหม่

Graph Introduction

วันที่ 4 เมษายน 2569 (เช้า)

ผู้ช่วยศาสตราจารย์เบญจมาศ ปัญญางาม

Outline

- Graph Definition
- Graph Applications
- Basic Graph Type
- Graph Representation
- Graph Implementation

กราฟ (Graph)

โครงสร้างข้อมูลแบบไม่เป็นเส้นตรง (Non-linear) ประกอบด้วย

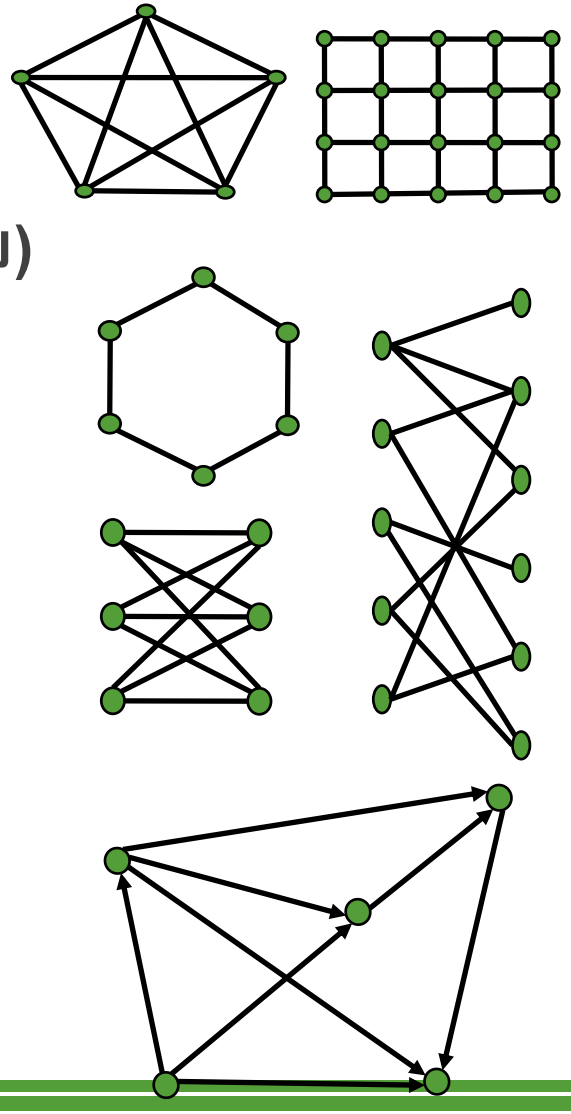
- **เซตของจุด (Vertices) หรือ nodes**
  - ✓ จุดบนกราฟที่ใช้แทนข้อมูล (เช่น สถานที่, คน, เมือง, หน้าเว็บ)

$$V = \{v_1, v_2, \ldots, v_n\}$$

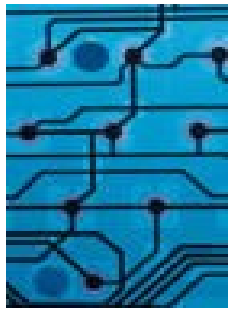
- **เซตของเส้นเชื่อม (Edges) ที่เชื่อมโยงระหว่างคู่**
  - ✓ เส้นที่เชื่อมต่อระหว่างคู่ Node ใด ๆ เพื่อแสดงความสัมพันธ์ (เช่น มีถนนที่เชื่อมถึงกัน, เป็นเพื่อนกัน, มีลิงค์หากันม,...)

$$E = \{e_1, e_2, \ldots, e_m\} \text{ เมื่อ } e_i = (v_j, v_k)$$

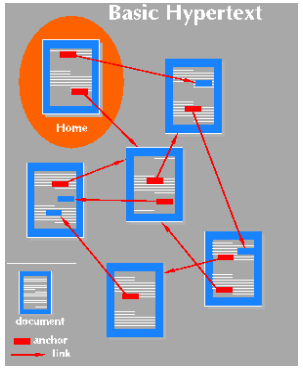
  - ✓ แต่ละเส้นเชื่อม e_i คือคู่ลำดับ (pair) ของ vertices $G = (V,E)$

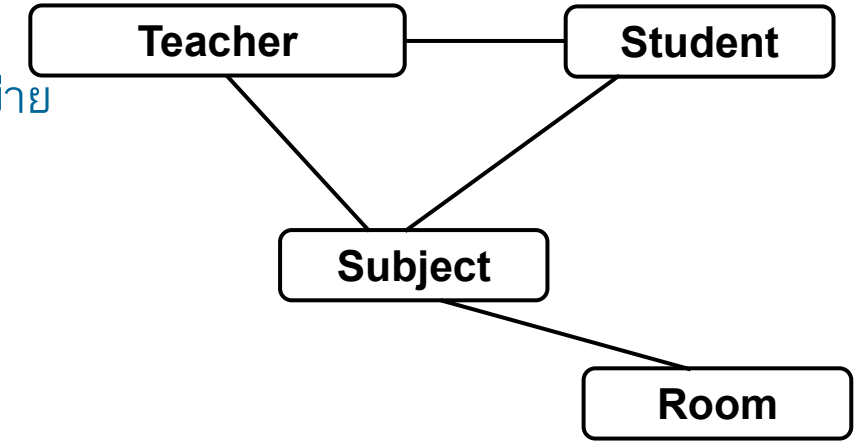


Applications of Graph



- Electronic circuits เครือข่ายไฟฟ้า :ค้นหาวงจรทั้งหมดในแผนผังวงจรไฟฟ้า
- Transportation networks เครือข่ายการขนส่ง :ค้นหาเส้นทางที่เร็วที่สุด
  - ✓ Road maps (e.g., Google maps)
  - ✓ Airline routes
- Computer networks
  - ✓ Local area network :ค้นหาเส้นทางที่สั้นที่สุดทั้งหมดระหว่างโหนดในเครือข่าย
  - ✓ Web pages with links เว็บไซต์และลิงก์เว็บ
  - ✓ Social Networks เครือข่ายสังคม :ค้นหาเพื่อร่วมกันในเครือข่าย
- Databases
  - ✓ Entity-relationship diagram





ตัวอย่างการประยุกต์

- การหา**เส้นทางที่สั้นที่สุด**และ/หรือประหยัดเวลาที่สุดในระบบ**แผนที่** สำหรับการขับรถยนต์จากเมืองไปยังอีกเมืองหนึ่ง
- การหา**เส้นทางที่เร็วที่สุด**การทำภารกิจของตัวละครใน **เกมคอมพิวเตอร์**
- การออกแบบคลองชลประทาน เพื่อหา**เส้นทางที่เหมาะสมที่สุด**เพื่อให้ได้เส้นทางที่มีปริมาณการไหลสูงสุดจากจุดต้นทางไปยังปลายทาง
- การออกแบบเครือข่ายทางรถไฟและถนนเพื่อ**เชื่อมต่อหลายเมืองเข้าด้วยกัน**

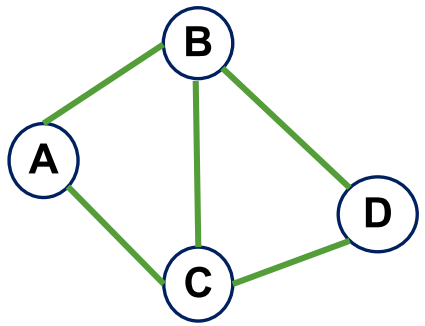
Basic Graph Type

- เส้นเชื่อมในกราฟจะเป็นเส้นเชื่อมแบบมีทิศทาง (Directed edge) หรือเส้นเชื่อมแบบไม่มีทิศทาง (Undirected edge) ก็ได้

(1) Undirected graph G = (*V*, *E*)

✓ E คือ เซตของเส้นเชื่อมแบบไม่มีทิศทางระหว่างคู่เวอร์เทคซ์ใด ๆ (u, v) เมื่อ u,v ∈ V (ปกติแล้ว u ≠ v นั่นคือ ไม่มี self-loops)

✓ โดยที่ (u,v) = (v,u)



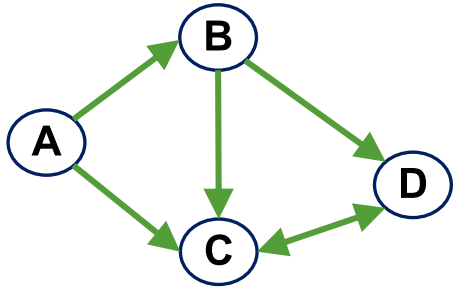
V = {A, B, C, D}

E = { (A, B), (A,C), (B,D), (B,C), (C, D) }

(2) Directed graph (digraph) G = (*V*, *E*)

✓ E คือ เซตของเส้นเชื่อมแบบมีทิศทางระหว่างคู่ลำดับ (Ordered pairs) ของเวอร์เทคซ์ใด ๆ (u, v) เมื่อ u,v ∈ V

✓ โดยที่ (u,v) ≠ (v,u)



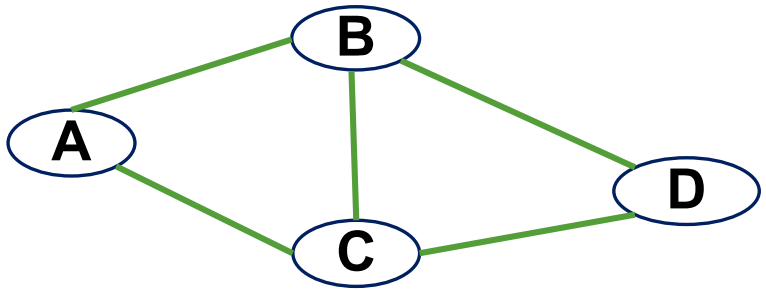
V = {A, B, C, D}

E = { (A, B), (A,C), (B,D), (B,C), (C, D) (D, C) }

Basic Graph Type

- ทุกกราฟแบบไม่มีทิศทางสามารถถูก convert ไปเป็นกราฟแบบมีทิศทาง ได้ง่าย ด้วยการแทนค่าแต่ละ undirected edge ด้วย directed edges แบบ 2 ทิศทาง ดังรูป

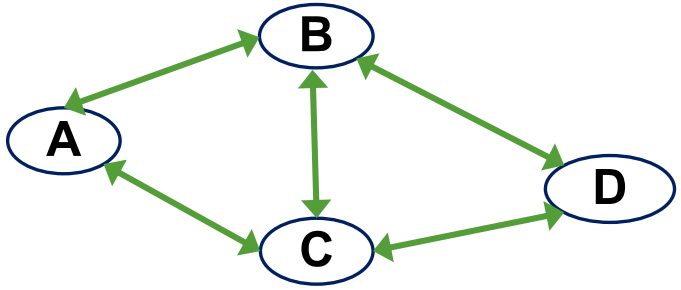
Undirected Graph



V = {A, B, C, D}

E = { (A, B), (A,C), (B,D), (B,C), (C, D) }

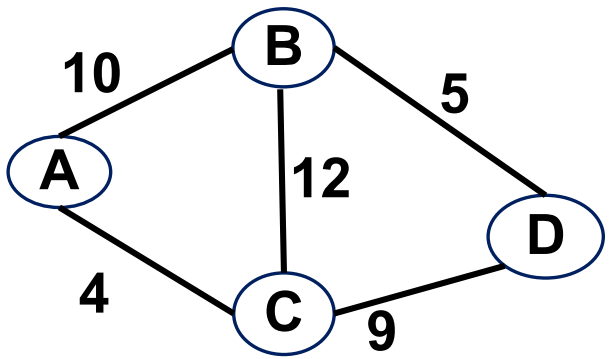
Directed Graph



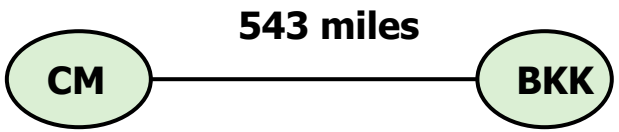
V = {A, B, C, D}

E = { (A,B), (B,A), (A,C), (C,A), (B,D), (D,B), (C,B), (B,C), (C, D), (D, C) }

Basic Graph Type

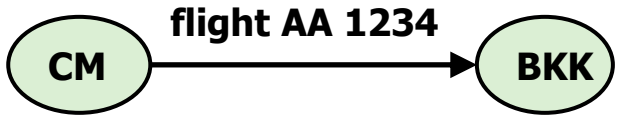


Weighted Graph

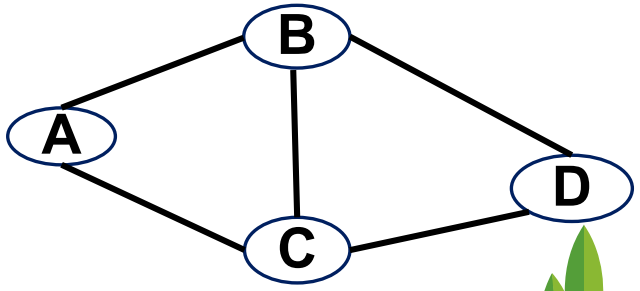


- ✓ แต่ละเส้นเชื่อมในกราฟมี **ค่าน้ำหนัก/ความสำคัญ** ของสัมพันธ์กับเส้นเชื่อมนั้น
- ✓ เช่น ระยะทางระหว่างเมือง, ปริมาณค่าใช้จ่ายในการเดินทางระหว่างสองสถานที่

Unweighted Graph

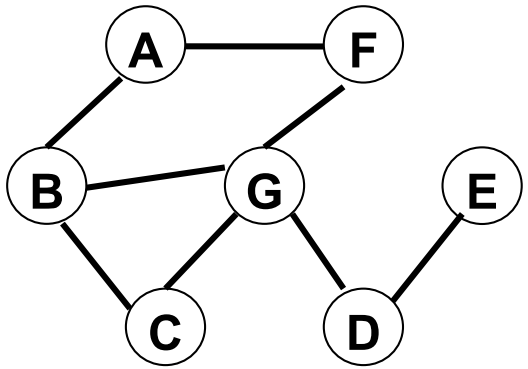


- ✓ ทุกเส้นเชื่อมมีความสำคัญเท่ากัน

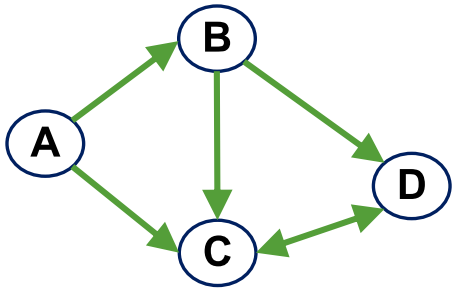


Graph Term

- **Adjacency** : u จะเป็น adjacent vertex ของ v
  เมื่อมี edge e = (u, v) | u, v ∈ V
  และ e ∈ E ในกราฟ G=(V,E)
  เช่น B และ F เป็น adjacent vertices ของ A



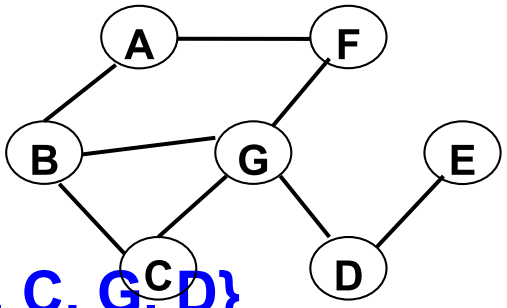
- **Degree** of a vertex: จำนวน edges ที่เชื่อมโยงไปยัง vertex
  - ✓ คือ จำนวนของ adjacent vertices เช่น degree ของ B คือ 3
  - ✓ กรณี directed graph แบ่งเป็น **In-degree** of a vertex และ **Out-degree** of a vertex เช่น
    - In-degree ของ B คือ 1
    - Out-degree ของ B คือ 2



Graph Term

- **Path** : sequence (หรือ ordered list) ของ vertices ในกราฟ
  เมื่อแต่ละ vertex จะมีเส้นเชื่อมไปยัง vertex ถัดไป

  เช่น { A, B, C, G, D} , { B, C, G, B}



- **Simple Path** : path ที่ไม่มีการซ้ำกันของ vertex ใน list เช่น { A, B, C, G, D}
- **Cycle** : Path ที่มี vertex เริ่มต้นและ vertex สุดท้าย เป็น vertex เดียวกัน
  และเป็น Simple path (list ของ vertices ไม่มี vertex ซ้ำกัน) เช่น { B, C, G, B}
- **Path length** :
  - ✓ใน Unweighted Graph : คือ จำนวน edge ทั้งหมดใน path
    เช่น ถ้าเดินทางจากจุด A -> B -> G -> D จะมีความยาวเส้นทางเท่ากับ 3
  - ✓ใน Weighted Graph : คือ ผลรวมน้ำหนักบนแส้นเชื่อม

Path and Path Length

Path Length

P1: 2 P3 : 5

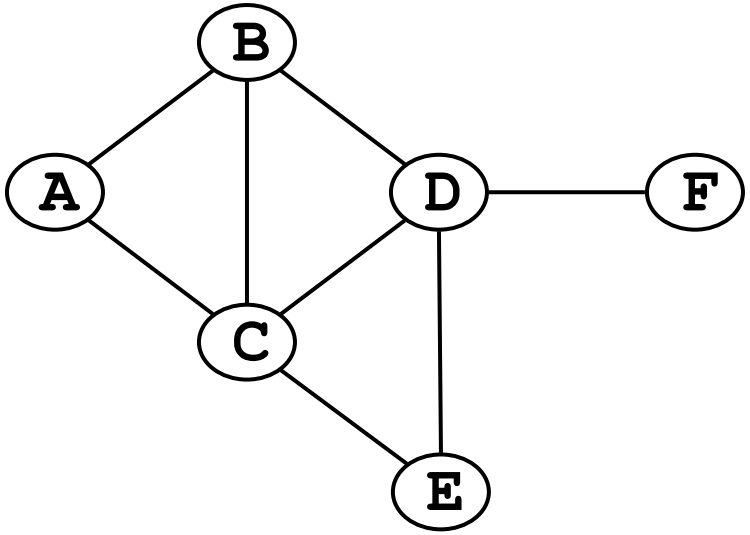
P2: 5 P4 : 6

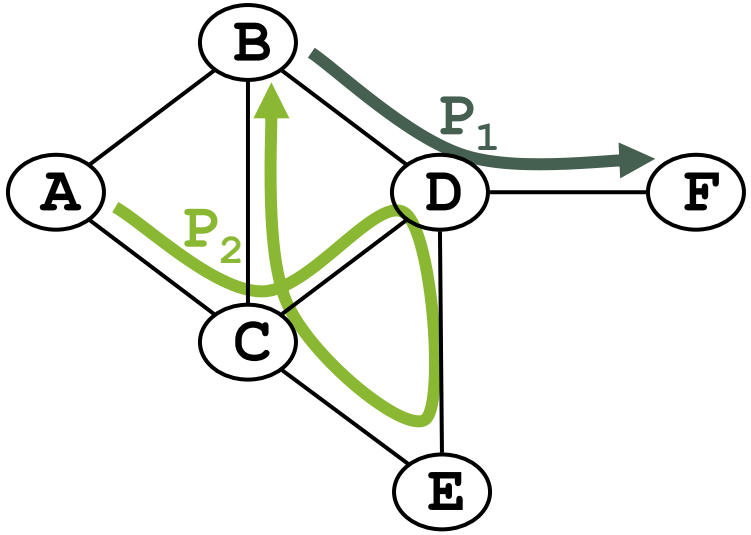
Simple Path : P1

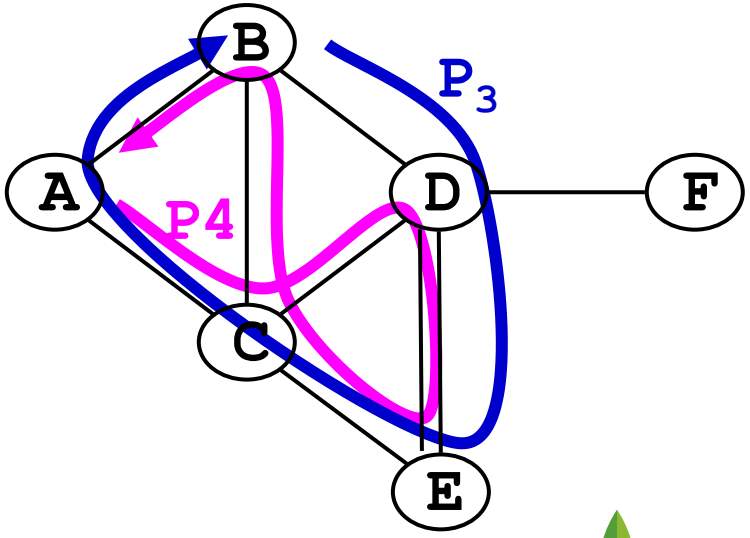
Path : P2

Cycle/Circuit : P3

Closed Walk : P4





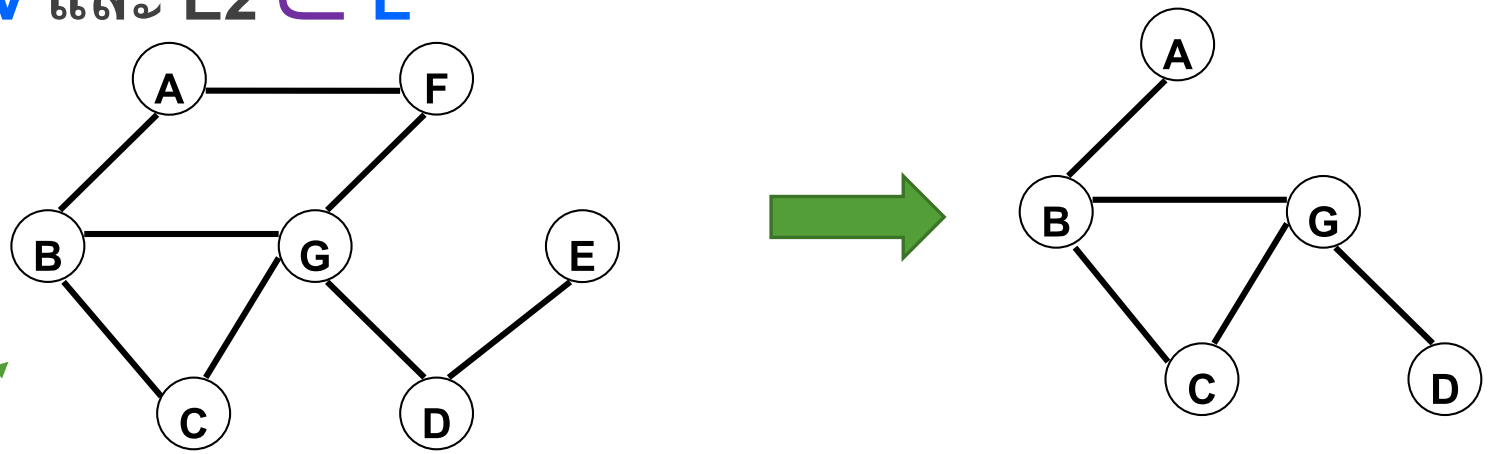


Hamiltonian Path : เส้นทางที่ลากผ่านทุกจุด (Vertex) ในกราฟเพียงครั้งเดียว

Eulerian Circuit : วงจรที่ผ่านทุกเส้นเชื่อมในกราฟ

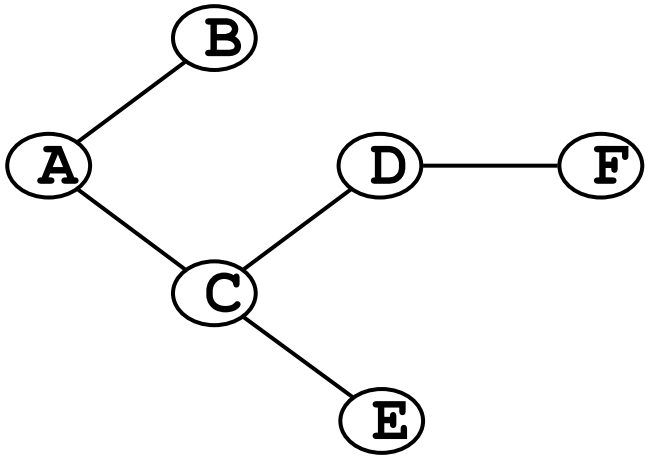
- **Subgraph** : G2=(V2,E2) ⊂ G=(V,E)

เมื่อ V2 ⊂ V และ E2 ⊂ E



- Cyclic Graph: กราฟที่มี Cycle

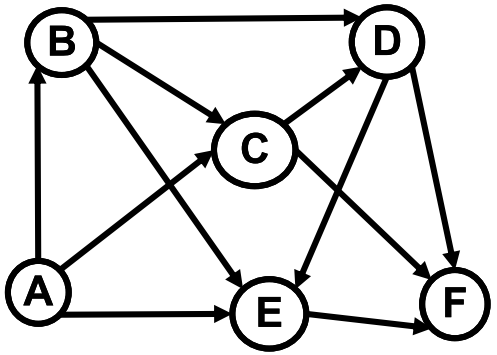
- Acyclic Graph: กราฟที่ไม่มีวงจรเลย
  - ✓ เช่น Tree หรือ Directed Acyclic Graph: DAG

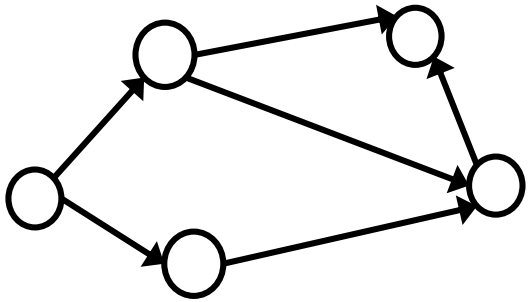


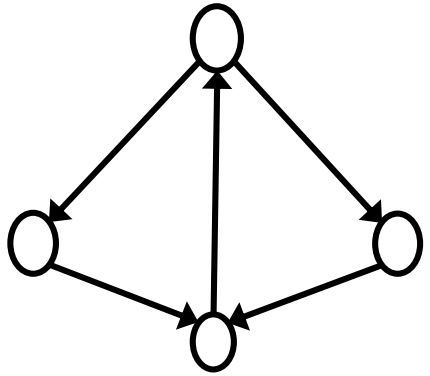
REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

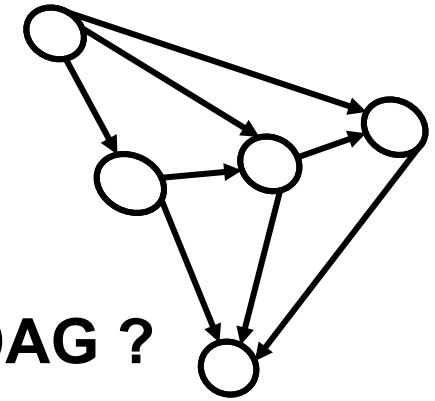
Directed Acyclic Graph (DAG)

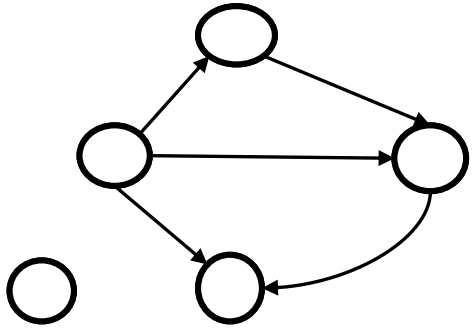
DAG เป็น Directed graph ที่ไม่มี Cycle ในกราฟเลย (คือไม่มี path ที่มี vetex เริ่มต้นและ vetex สุดท้ายใน list ที่เป็น vetex เดียวกัน)

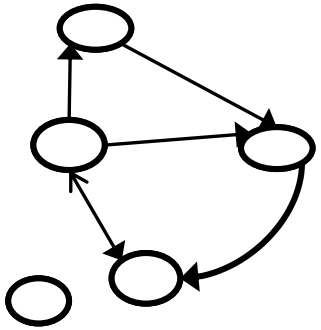








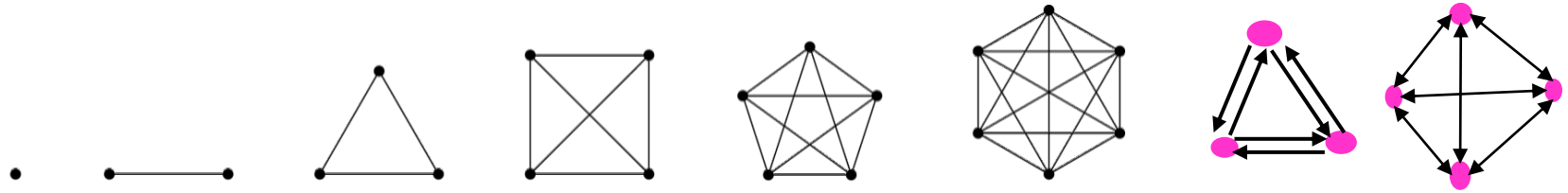




กราฟใดเป็น DAG ?

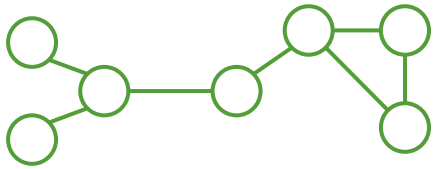
Connected Graph

- **Complete graph (or fully connected or strongly connected)**
  - ทุกคู่ของ vertices (u,v) จะมีเส้นเชื่อม (edge) จาก u ไป v

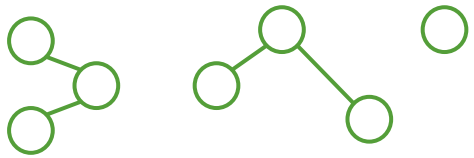


➢ **Connected graph**

✓ ทุกคู่ของ vertices (u,v) มีอย่างน้อย **1 path** จาก u ไป v



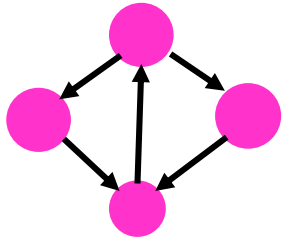
Connected graph



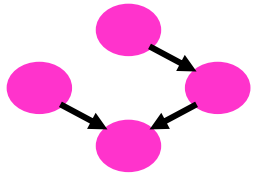
Disconnected graph

Directed-Graph Connectivity

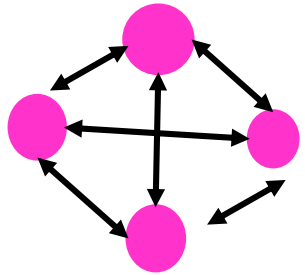
- **Strongly connected directed graph**
  - ✓ ทุกโหนดจะมี 1 path ที่เชื่อมโยงไปยังโหนดอื่น ๆ ทุกโหนด



- **Weakly connected directed graph**
  - ✓ ทุกโหนดจะมี 1 path ที่เชื่อมโยงไปยังโหนดอื่น ๆ ทุกโหนด โดยไม่สนใจทิศทางของเส้นเชื่อม



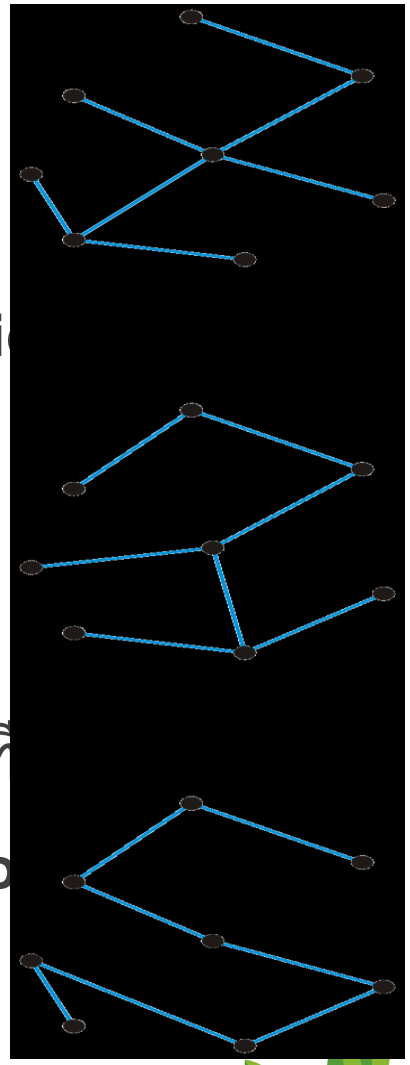
- **Fully connected directed graph**
  - ✓ ทุกโหนดจะมีเส้นเชื่อม (edge) ที่เชื่อมโยงไปยังโหนดอื่น ๆ ทุกโหนด



Tree & Graph

กราฟ G = (V,E) ใด ๆ จะเป็นทรีได้

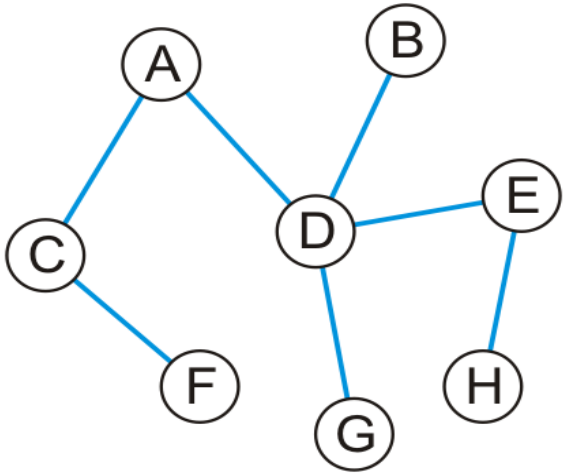
- ถ้าเป็น Connected graph ที่มี path ระหว่างคู่ vertices ใด ๆ แบบ uni
- โดยที่กราฟ G จะมีคุณสมบัติดังนี้
  - ✓ มีจำนวนของ edges ($|E|$) = $|V| - 1$
  - ✓ เป็น *acyclic* graph (ไม่มี cycle ในกราฟ)
  - ✓ หากเพิ่ม edge อีกเส้นเชื่อมหนึ่งลงในกราฟ G อีกจะเกิด cycle ทันที
  - ✓ หากมีการลบ edge หนึ่ง edge ใดในกราฟ G จะเกิด two disjoint no sub-graphs



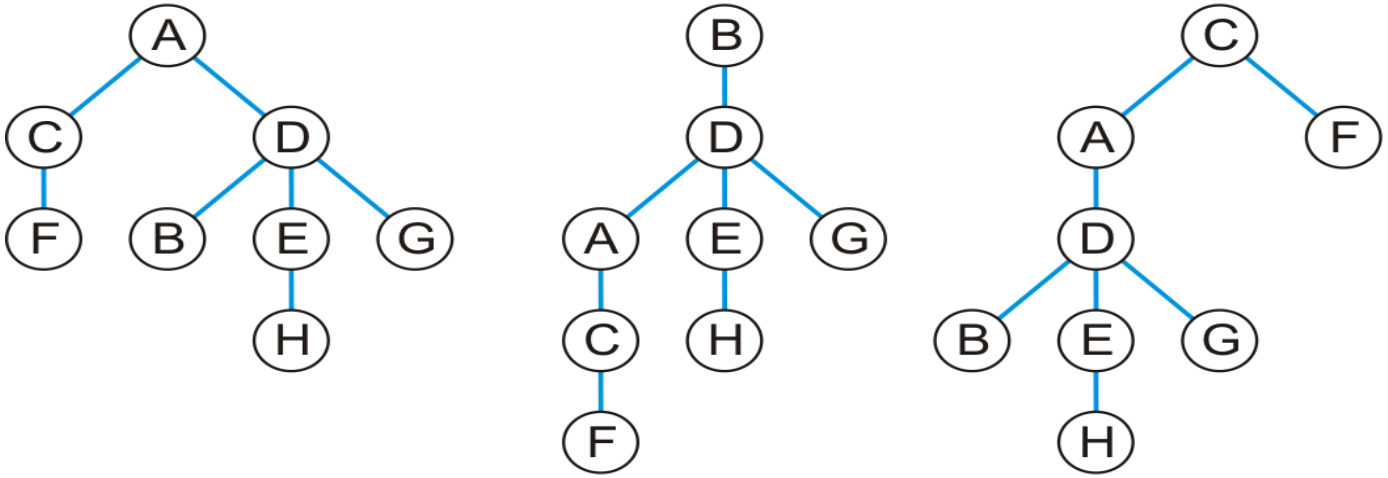
Graph & Tree

วิธีการ Convert จาก= (V,E) เป็น rooted tree

- เลือก vertex ใด vertex หนึ่งในกราฟ เป็นโหนดราก (root)
- กำหนด neighboring vertices ของโหนดรากที่เลือกนี้เป็น children ของโหนดราก มีสถานะ parent - child
  - จากนั้นกำหนดสถานะ parent – child ให้ครบทุกโหนดในกราฟ



กราฟ G

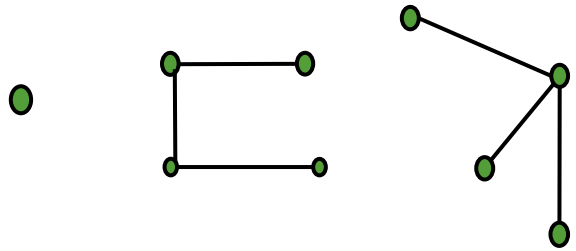


ตัวอย่างทรี 3 แบบที่เลือกโหนด A B และ C เป็น root ตามลำดับ

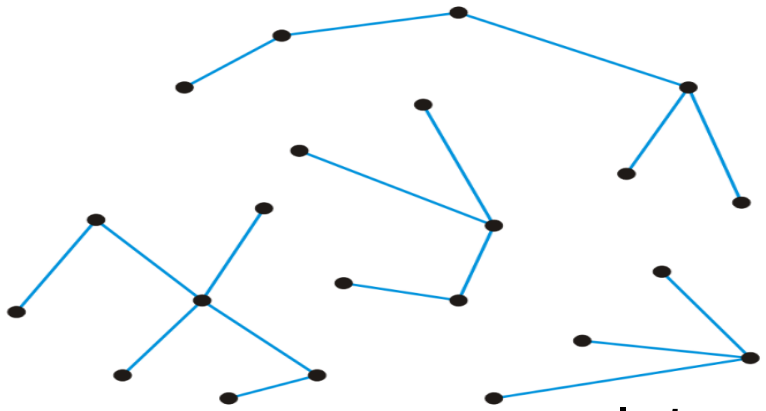
Graph & Forests

คือกราฟใด ๆ ที่ไม่มี cycle และมีคุณสมบัติดังนี้

- จำนวน edges (|E|) < จำนวน vertices (|V|)
- จำนวนทรีเท่ากับ |V| – |E|
- หากมีการลบ edge หนึ่ง edge ใดในกราฟ G จะเป็นการเพิ่มจำนวนทรีใน Forest



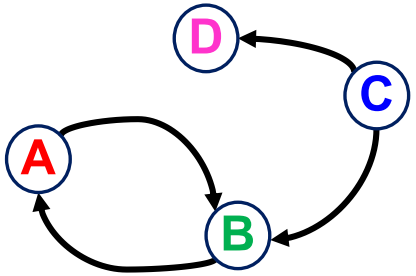
ตัวอย่างกราฟ G= (V,E) ที่เป็น Forest
โดย |V| = 9 , |E| = 6 ประกอบด้วย 3 ทรี



ตัวอย่างกราฟ G= (V,E) ที่เป็น Forest
โดย |V| = 22 , |E| = 18 ประกอบด้วย 4 ทรี

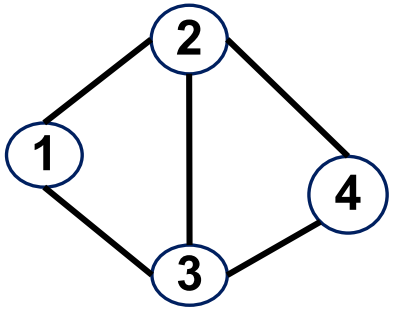
Representations of Graphs

➢ วิธีการเก็บข้อมูล



$n = 4 = |V|$,

$m = 4 = |E|$



$n = 4 = |V|$,

$m = 5 = |E|$

Adjacency matrix:

	A	B	C	D
A	0	1	0	0
B	1	0	0	1
C	0	1	0	1
D	0	0	0	0

	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	1
4	0	1	1	0

Adjacency list:

A → B → \

B → A → \

C → B → D → \

D → \

1 → 2 → 3 → \

2 → 1 → 3 → 4 → \

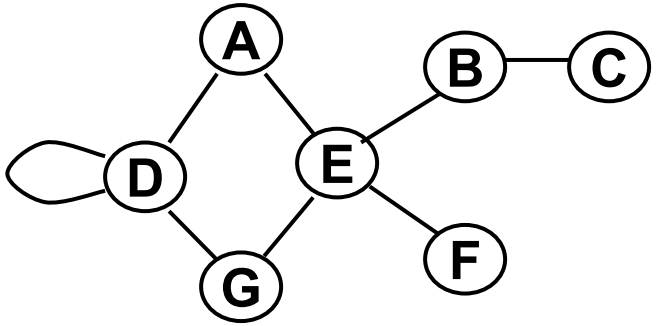
3 → 1 → 2 → 4 → \

4 → 2 → 3 → \

Adjacency-matrix representation

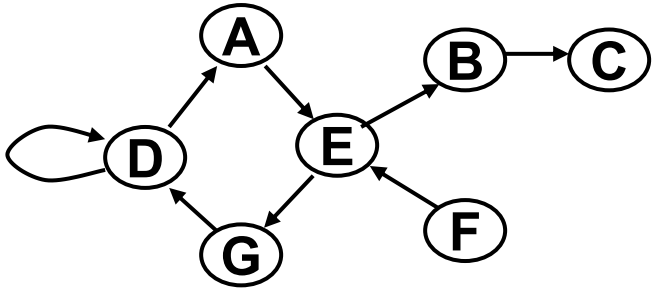
- The adjacency matrix is symmetric about the main diagonal

ตัวอย่าง



	A	B	C	D	E	F	G
A				●	●		
B			●		●		
C		●					
D	●			●			●
E	●	●				●	●
F					●		
G				●	●		

- An adjacency matrix can equally well be used for digraphs (directed graphs)



	A	B	C	D	E	F	G
A					●		
B			●				
C							
D	●			●			
E		●					●
F					●		
G				●			

Implementations of Adjacency Matrix

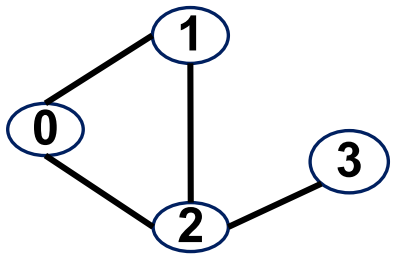
```
#include <iostream>
using namespace std;

int main() {
  int V = 4; // จำนวนโหนด
  int adjMatrix[4][4] = {0}; //เริ่มต้นด้วย 0 ทั้งหมด

  // เพิ่มเส้นเชื่อมระหว่างโหนด (0-1, 0-2, 1-2, 2-3)
  adjMatrix[0][1] = adjMatrix[1][0] = 1;
  adjMatrix[0][2] = adjMatrix[2][0] = 1;
  adjMatrix[1][2] = adjMatrix[2][1] = 1;
  adjMatrix[2][3] = adjMatrix[3][2] = 1;

  for(int i = 0; i < V; i++) {
      for(int j = 0; j < V; j++)
          cout << adjMatrix[i][j] << " ";
      cout << endl;
  }
  return 0;
}
```

ผลรัน
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

```
// undirected graph
//|V| = 4
//|E| = 4
```

➢ ถ้าประกาศแบบ Static Array (ระบุขนาดคอลัมน์ตายตัว) ต้องระบุขนาดคอลัมน์ ที่ argument ของฟังก์ชันด้วย

```
void display( int adjM[][4], int V){
 for(int i = 0; i < V; i++) {
    for(int j = 0; j < V; j++)
        cout << adjM[i][j] << " ";
    cout << endl;
  }
}
```

Implementations of Adjacency Matrix

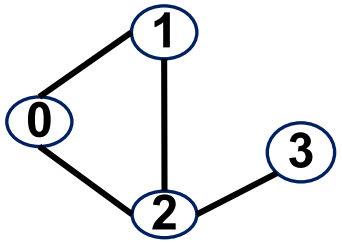
➢ กรณีใน main() ระบุแบบ Dynamic Array

```
#include <iostream>
using namespace std;

int main() {
   int V = 4; //  จำนวนโหนด
   //int adjMatrix[4][4] = {0}; //เริ่มต้นด้วย 0 ทั้งหมด

   int** adjMatrix = new int*[V];
   for(int i = 0; i < V; i++)
        adjMatrix [i] = new int[V]{0}; //กำหนดค่าเริ่มต้นเป็น 0

   //เพิ่มเส้นเชื่อมระหว่างโหนด
   addEdge(adjMatrix, 0, 1);
   addEdge(adjMatrix, 0, 2);
   addEdge(adjMatrix, 1, 2);
   addEdge(adjMatrix, 2, 3);
   display(adjMatrix,V);
   return 0;
}
```




```
// undirected graph
//|V| = 4
//|E| = 4
```

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

➢ การระบุ argument ที่ฟังก์ชันไม่ต้องระบุจำนวนคอลัมน์

```
void display(int* adjM[], int V) {
//หรือ void display( int** adjM, int V){
  for(int i = 0; i < V; i++) {
    for(int j = 0; j < V; j++)
        cout << adjM[i][j] << " ";
    cout << endl;
  }
}
```

```
void addEdge (int* adjM[], int startVertex, int endVertex){
  adjM[startVertex] [endVertex] = 1;
  adjM[endVertex] [startVertex] = 1; //ตัดทิ้งถ้าเป็น Directed graph
}
```

ผลรัน

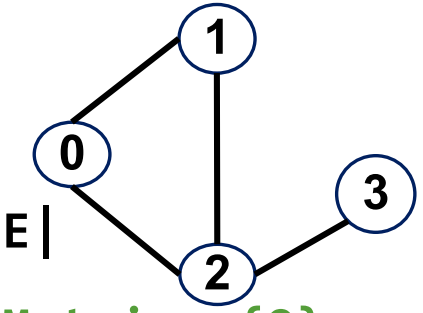
```
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
```

Implementations of Adjacency Matrix

➢ กรณี adjMatrix ใช้ตัวแปร Global

```
int **adjMatrix;
int numEdges;
void initializeGraph (int n) {
    adjMatrix = new int *[n];
    for(int i = 0; i < n; i++)
        adjMatrix [i] = new int[n]{0};
    }
    numEdges = 0;
}
```

```
//numVertices = |V|
int numV;
int numEdges = 0; //|E|
int [numV][numV]  adjMatrix ={0}
```



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

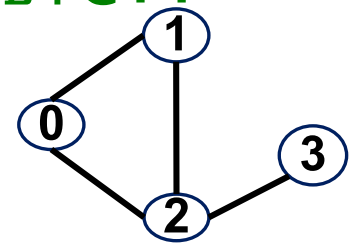
```
void addEdge (int startVertex, int endVertex) {
    adjMatrix[startVertex] [endVertex] = 1;
    adjMatrix[endVertex] [startVertex] = 1; //ตัดทิ้งถ้าเป็น Directed graph
    numEdges++; //นับ edge เอง
}
```

Implementations of Adjacency Matrix

➢ กรณีใช้ vectors ใน STL ไลบารี ของภาษา C++

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
  int V = 4; // จำนวนโหนด
  vector<vector<int>> adj(V, vector<int>(V, 0));
  addEdge(adj, 0, 1);
  addEdge(adj, 0, 2);
  addEdge(adj, 1, 2);
  addEdge(adj, 2, 3);
  display(adj,V);
  return 0;
}
```




```
// undirected graph
//|V| = 4
//|E| = 4
```

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

ผลรัน

0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0

```
void addEdge(vector<vector<int>> &adj,
             int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1;
}
```

```
void display(vector<vector<int>> adj, int V)
```

กรณีฟังก์ชันไม่ pass by reference (&) จะได้ผลรัน เป็น

0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

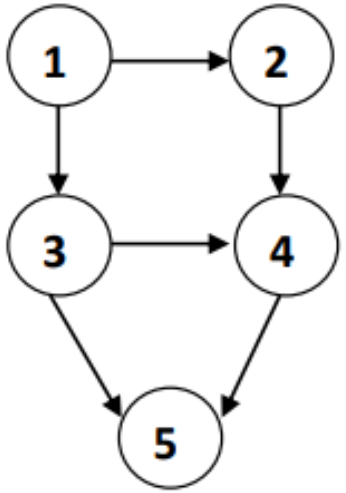
อาจมีการเช็ค ว่า

u >= 0 && u < V && v >= 0 && v < V

Implementations of Adjacency Matrix

- กรณีใช้ vectors ใน STL ไลบารี ของภาษา C++ : ตัวอย่างการประกาศ
  - ✓ `vector<vector<int>> adj(V, vector<int>(V, 0));`
  - ✓ `vector<vector<bool>> adj(V, vector<int>(V, false));`
  - ✓ `vector<vector<int>> adj(n + 1, vector<int>(n + 1, -1));`
- สามารถใช้ .size() หาขนาดเอง โดไยไม่ต้องเก็บค่า n
- เพิ่มขนาดได้ง่าย โดยใช้ .push_back()
- การคืนหน่วยความจำทำให้อัตโนมัติ
  - ✓ วิธี Dynamic array (แบบ Pointer based) จะต้องทำ delete เอง
- ข้อเสีย คือ Memory Overhead: ใช้หน่วยความจำเยอะกว่าเล็กน้อยในการเก็บโครงสร้าง และทำงานช้ากว่า Pointer เล็กน้อย

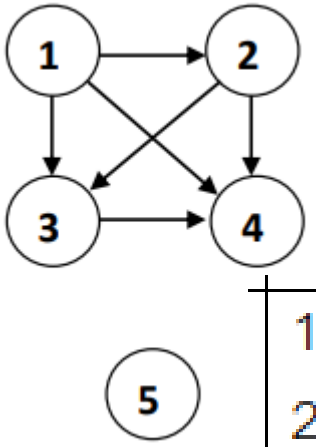
C2P29:In-Degree และ Out-Degree ของแต่ละ Vertex



ตัวอย่าง

ข้อมูลเข้า
5 6
1 2
1 3
2 4
3 4
3 5
4 5

ข้อมูลออก
1->2->3
2->4
3->4->5
4->5
5
0 2
1 1
1 2
2 1
2 0



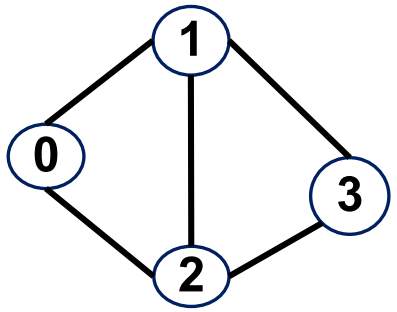
5 6
1 2
1 3
1 4
2 3
2 4
3 4

1->2->3->4
2->3->4
3->4
4
5
0 3
1 2
2 1
3 0
0 0

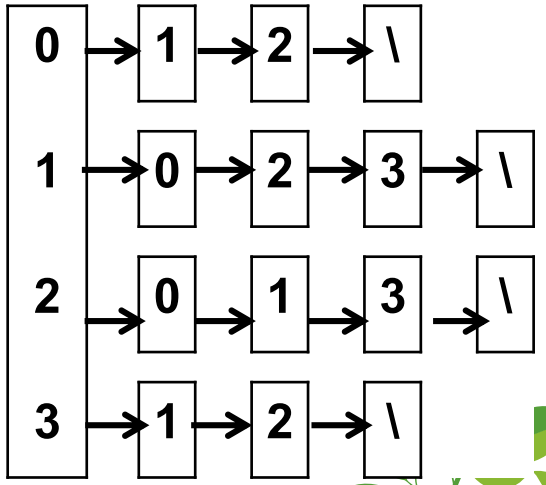
- รับจำนวนโหนด (V) และจำนวน Edges แล้วรับเส้นเชื่อมแต่ละ Edge
- Output ให้แสดงกราฟและจำนวนดีกรีแต่ละโหนด

Implementations of Adjacency Lists

- ใช้อาร์เรย์ของ linked lists
  - ✓ ต้องระวังการจัดการเกี่ยวกับ pointer และเข้าใจการใช้งาน linked list
- ใช้ vectors ใน STL ไลบารี ของภาษา C++
- กำหนด **array of vectors** เพื่อทำงานในลักษณะ **adjacency list**




```
vector<int> adj[n]; //#include <vector>
addEdge(adj, 0, 1);
```




```
void addEdge(vector<int> adj[], int u, int v) {
    adj[u].push_back(v); // Add v to adjacency list of u
    adj[v].push_back(u); // Add u to adjacency list of v
}
```


REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Implementations of Adjacency Lists

```
#include <bits/stdc++.h>
//#include <iostream>
//#include <vector>
using namespace std;

int main() {
   int V = 5; // จำนวนโหนด
    vector<int> adj[V];

    addEdge(adj, 0, 1);
    addEdge(adj, 0, 4);
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 1, 4);
    addEdge(adj, 2, 3);
    addEdge(adj, 3, 4);

    display(adj, V);
    return 0;
}
```

```
void display(const vector<int> adj[], int V){
   for (int v = 0; v < V; ++v) {
      cout << "Node " << v << ": ";




      cout << endl;
   }
}
```

ผลรัน

```
Node 0: -> 1-> 4
Node 1: -> 0-> 2-> 3-> 4
Node 2: -> 1-> 3
Node 3: -> 1-> 2-> 4
Node 4: -> 0-> 1-> 3
```

- แนะนำให้ฟังก์ชัน display() ควรกำหนด **const**
- ใช้ในกรณีดึงข้อมูลไปคำนวณหรือแสดงผล โดยห้ามแก้ไขค่า

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Implementations of Adjacency Lists

การประกาศเพื่อเก็บค่า weighted graph

```
➢struct Edge {
   int v;//target
   int weight;
};
```

และ `vector<Edge> adj[n];`

หรือ

➢`vector<pair<int,int>> adj[n];`

➢คำสั่งต่าง ๆ

- ✓`adj[u].push_back({v, weight});`
- ✓`for (auto& edge : adj[u])`
- ✓`edge.v` หรือ `edge.first`
- ✓`edge.weight` หรือ `edge.second`

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Other Implementations

- map<string, vector<string>> adj;
- map<string, vector<pair<string, int>>> adj;
  - ✓ adj["A"].push_back({"B", 5});
  - ✓ for (auto const& edge : edges)
  - ✓ edge.first
  - ✓ edge.second

```
struct Edge {
    string to;
    int weight;
};
```

- map<string, vector<Edge>> adj;
- map<string, set<string>> adj; //ต้องการให้เรียงลำดับอัตโนมัติ+ ไม่มีข้อมูลซ้ำ
- map<string, set<pair<string, int>>> adj;
- vector<vector<pair<int, int>>> แทน map จะทำงานได้เร็วกว่า